Challenge I

Today

Welcome to your next lesson: Challenge I.

I'm preparing your lesson based on what you've learned so far, what comes next, and how to build your skills step by step.

Keep it up, every step counts.

Ron, let’s bring everything together into a small library program today 📚

Imagine this tiny story:

There is a Reader with a name.
There is a Book with a title and an available status (like True for on-shelf, False for checked out).
The Reader can try to borrow the Book.
When they do that, the Reader uses one of their own methods, which then calls a method on the Book.

Visually:

Text
```
reader ——uses——> book
   |               |
 name          title, available
   |               |
borrow() ——calls——> check_out()
```
To go step by step, I want you to explain the diagram in your own words (no code yet):

Who should be one class?
Who should be the other class?

What kind of information is stored in each one (attributes)?
What is the action each one can do (methods)?

You can answer in a few short sentences, like:

“Reader is a class that has ..., Book is a class that has ...”

a reader which has a name uses a book by checking if the title is available ot not and if so uses its own borrow method to checkout the book

Nice description, Ron 👍 You’ve got the big idea.

Let me tighten it slightly, step by step:

Reader class

Attribute: name
Method: borrow()
This method will use a Book object (ask the book if it can be checked out).
Book class

Attributes: title, available
Method: check_out()
This method will change or report the book’s availability.

So the flow is:

Text
Reader.borrow() ---> calls ---> Book.check_out()
Next step: we’ll turn this into real Python code with two classes that interact.

Before we do that, are you okay if:

Reader.borrow() does not directly check the title,
but instead calls book.check_out() and then prints a message based on what the book says?

Answer with just “yes” or “no”, and we’ll build it in the editor next.

Yes, that makes sense

Great, we'll keep it like that 👌

Let's now sketch the two classes in words, then we'll move to the code editor.

I want this basic structure:

A Book class with:
an attribute for the title (like title)
an attribute for whether it's available (like available)
a method check_out(self) that:
looks at self.available
if it's available, it marks it as not available and maybe returns something (like True)
if it's not available, it returns something else (like False)

We'll wire Reader.borrow() to use this.

Before we go to the editor, can you tell me in plain words:

When book.check_out() runs, what do you think it should do step by step with available?

call the bookinstance.checkout and see if the result is true or false, if true check it out

Exactly 👌 You're thinking in the right direction.

Let me lay it out clearly:

For Book.check_out() itself, inside the Book:

Look at self.available.
If self.available is True:
set self.available to False (book is now taken)
return True (checkout succeeded)
Else:
return False (someone already has it)

Then Reader.borrow() will call book.check_out() and react to that True/False.

Let’s move this into code now.

In the code editor, start with just the Book class:

Open the editor (</> button).
Write a Book class with:
no __init__
a check_out(self) method that follows the 3 steps above.
At the bottom, create one Book object (like book1 = Book()) and after that, set:
book1.title to some string
book1.available to True
Call book1.check_out() once, and print() the result.

Run and then Submit code when you’re done.

I’ll check it and then we’ll add the Reader class next 👣

Python
```
class Book:
    def check_out(self):
        if self.available:
            self.available = False
            return True
        else:
            return False

book1 = Book()
book1.available = True
print(f'{book1.check_out()}')
```
Nice work Ron — your Book class and check_out logic are spot on 🎯

You created the object, set available, and the output True matches the first checkout.

Right now, the book can manage itself. Next, we’ll let a reader use it.

We’ll add a Reader class that:

```
168
169   has a name attribute (set after creation, like you did with
      available)
170   has a method borrow(self, book) that:
171   calls book.check_out()
172   checks the True/False result
173   prints a message like:
174   if True: "Ron borrowed <title>"
175   if False: "<title> is not available"
176
177   Does this step-by-step behavior for borrow(self, book) make
      sense before you code it?
178
179   Yes, that makes sense
180
181   Perfect 🌟
182
183   Let’s extend your program step by step.
184
185
186   In the code editor, update your code like this:
187
188
189   Keep your existing Book class as-is.
190   Add a new Reader class:
191   with a method borrow(self, book) that:
192   calls book.check_out()
193   if it returns True, print(f"{self.name} borrowed
      {book.title}")
194   else, print(f"{book.title} is not available")
195   At the bottom:
196   keep book1 = Book()
197   set book1.title and book1.available = True
198   create a Reader object, e.g. reader1 = Reader()
199   set reader1.name to your name
200   call reader1.borrow(book1) twice in a row.
201
202   Run, then Submit code so I can see both the code and the
      output.
203
204   Python
205   class Book:
206       def check_out(self):
207           if self.available:
208               self.available = False
209               return True
210           else:
```

```
            return False
class Reader:
    def borrow(self, book):
        if book.check_out():
            print(f'{self.name} borrowed {book.title}')
        else:
            print(f'{book.title} is not available')

book1 = Book()
book1.title = "The Hobbit"
book1.available = True
reader1 = Reader()
reader1.name = "Ron:"
reader1.borrow(book1)
reader1.borrow(book1)
Ron, this is beautiful work 🎯


You've got exactly what we wanted:


Two classes: Book and Reader
Attributes set after creation: title, available, name
Interaction flow:
reader1.borrow(book1) calls book1.check_out()
The first call prints: Ron: borrowed The Hobbit
The second call prints: The Hobbit is not available
The book's state (available) changes correctly after the
first checkout.

So the call chain is:

Text
reader1.borrow(book1)
   → inside Reader.borrow: calls book1.check_out()
       → inside Book.check_out: uses self.available, sets it
to False, returns True/False
   → back in Reader.borrow: prints the right message
Quick check-in 🔍


In your own words (short and simple):

When reader1.borrow(book1) runs the first time, which
object's method runs first, and which object's method runs
second?

```

it runs reader1s borrow method first, which callls the second method to run, books check_out method

Exactly right ✅

First: reader1.borrow(book1) runs on the Reader object.
Inside that, it calls book1.check_out(), which runs on the Book object.

So the flow is:

Text
Reader.borrow → Book.check_out → back to Reader.borrow (print)
Let’s test your understanding with a small change in the story.

Imagine we add a second book:

```
book2 = Book()
book2.title = "Dune"
book2.available = False
```

Then we call:

Python
```
reader1.borrow(book2)
```
Question for you (just think it through, or answer briefly):

What exact message do you expect to be printed for that call?

Dune is not available

Exactly 🎯

Because:

book2.available is False
book2.check_out() returns False
So Reader.borrow goes to the else branch and prints:
Dune is not available

You've just correctly predicted an interaction between two objects from different classes. That's core OOP thinking.

I'd like to zoom out for a second:

You now have a full little app with:

Book class (data + behavior)
Reader class (data + behavior)
One object calling another object's method and reacting to its return value.

To be sure you fully own this, pick one of these to explain in 1–2 sentences:

A) What self refers to inside Book.check_out.

B) What self refers to inside Reader.borrow.

Just choose A or B and explain it in your own words.

a: self refers to the object book1 in this case, the unique values of the data like available,which is differnt from another book object created like book2

Yes Ron 🙌 that's exactly it.

Inside Book.check_out, self is:

"the specific Book object this method is running on right now"

So when you call book1.check_out():

self → book1
self.available → book1.available

If you called book2.check_out() instead, then:

self → book2
self.available → book2.available

Each object keeps its own state.

You've completed this challenge:

You wrote a small Python program with real-world classes (Book, Reader) ✅
You made objects interact through method calls and attributes ✅
You traced who calls what, and what self refers to ✅

Mission complete on this lesson 🚀

When you're ready, you can head to the next lesson in the Learn tab, or if you want, tell me what part of OOP you'd like to get even stronger at next (methods, objects, modeling real systems, etc.).

```
class Book:
    def check_out(self):
        if self.available:
            self.available = False
            return True
        else:
            return False
class Reader:
    def borrow(self, book):
        if book.check_out():
            print(f'{self.name} borrowed {book.title}')
        else:
            print(f'{book.title} is not available')
```